

PSF: A Generic and Extensible Framework for Protocol State Fuzzing

Konstantinos Sagonas^{1,3}  and Thanos Typaldos^{2,3} 

¹ Uppsala University, Uppsala, Sweden
kostis@it.uu.se

² Yale University, New Haven, USA

³ National Technical University of Athens, Athens, Greece
actypaldos@gmail.com

Abstract. In recent years, protocol state fuzzing has emerged as an effective technique to analyze and test network protocol implementations, uncovering numerous security vulnerabilities, bugs, and non-conformance issues in them. This paper presents PROTOCOLSTATE-FUZZER (PSF), an open source, generic, modular, and extensible framework for state machine learning and testing of network protocol implementations. We describe the distinctive features and support that PSF offers, its architecture and implementation, and briefly overview the protocol-specific state fuzzers that currently use PSF as their basis.

Keywords: Analysis and testing of network protocol implementations
· Model learning · Model-based testing · Protocol state fuzzing

1 Introduction

Implementations of network and security protocols are core components of critical infrastructures of our society. Consequently, their correctness and robustness are of paramount importance. The specifications of such protocols (e.g., their RFCs) prescribe, usually in natural language, the interaction between the protocol peers, the format of data to be exchanged, and the actions that each peer needs to take at different protocol stages. In other words, specifications implicitly outline the underlying state machine of the protocol. On the other hand, a specific implementation realizes a variant of this implicit state machine, which needs to be as close to the protocol specification as possible. Any deviation between the two can lead to a harmless error at best, to interoperability and denial-of-service errors, or to an exploitable vulnerability in the worst case. The questions then become: “How can state machines of different implementations be obtained so that they can be verified against the specification? How can they be tested for non-conformance or for detecting deviant behaviors in them?”

One way to answer these questions is through *learning*. There exist (black-box) techniques that can automatically infer a close approximation of a given implementation’s state machine, usually regarding its input/output behavior.

In the formal methods community, these techniques have appeared under different names, such as the *Minimally Adequate Teacher framework* [1], *active automata learning* [27,36], *model learning* [38], and, in the context of testing network protocol implementations, *protocol state fuzzing* [31,9], which is the name we will use from now on in this paper. The output of these techniques is a state machine model that can be used to analyze the implementation’s behavior and check whether it complies with its protocol specification, or, if not, detect bugs and non-conformance issues in it either by ocular model inspection [31,6,9], semi-automatically by detecting differences in the learned models [2,8,18], automatically by automata-based techniques [12,13], or by model checking [25,10].

There exist publicly available libraries that can be used for automata learning, such as LearnLib [20,15] or AALpy [24]. However, for a protocol state fuzzing tool, some additional components are needed: (1) a *learning component*, (2) a *mapping component*, and (3) a *system component*. The learning component drives the learning library by sending queries (sequences of abstract input messages) to and receiving answers (sequences of abstract output messages) from the mapping component. The mapping component translates abstract messages to/from concrete ones that are understood by the System Under Learning (SUL), which is managed by the system component. Alternatively, the learning component can be swapped out by (4) a *testing component*, while most of the remaining infrastructure can be used as is. In this way, a protocol state fuzzing tool can also be used for testing or fuzzing the System Under Testing (SUT).⁴

This paper presents PROTOCOLSTATE-FUZZER (PSF, for short): a generic, modular, and extensible framework that can be used both for learning and testing of network protocol implementations. It provides the four components mentioned above as well as the support which is needed for optimization or to address practical problems. The following sections give more details about the requirements and challenges of protocol state fuzzing (Section 2), present PSF’s design and implementation (Sections 3 and 4), and briefly mention protocol analysis and testing tools that are currently using it as their basis (Section 5). The paper ends with related work and some concluding remarks (Sections 6 and 7).

2 Protocol State Fuzzing: Requirements and Challenges

The two main modes of using a protocol state fuzzing tool, and thus of PSF, are *learning* and *testing* a SUL/T. For reasons that we will explain soon, PSF also supports a third mode: *timing*. In this section, we present requirements for supporting these modes, and some challenges of practical nature that they involve. We start with a paragraph reviewing state machine model learning.

Model learning occurs through several cycles of *hypothesis construction* and *hypothesis validation*. Hypothesis construction builds a state machine consistent with prior exchanges; this state machine is called the *hypothesis* H . Hypothesis validation then checks if H indeed correctly describes the SUL, e.g., by doing

⁴ We use SUL to refer to the protocol implementation whose state machine we learn, SUT when we test/fuzz it, and SUL/T to refer to it in general.

model-based testing. If the answer is “yes”, H is returned as the final state machine model. If it is “no”, a counterexample C exposing the inconsistency is provided to hypothesis construction, which will then use this C to produce a refined hypothesis H' . The process continues until hypothesis validation answers “yes”, meaning it could not find a counterexample in the last hypothesis within some specified limit (time, number of tests, etc.). A crucial assumption by many/most automata learning algorithms is that the SUL is *deterministic*; otherwise, the above process may not terminate or may produce inaccurate state machine models. For learning state machines of protocol implementations, we need a mediating component, called *mapper*, which translates between the input/output symbols that the learning component uses, and the concrete messages that the SUL understands. In order to produce correct messages, the mapper typically needs to maintain the state of the interaction, e.g., the encryption keys necessary to produce messages that the other peer can decrypt. The learned models depend on the symbol-to-message abstraction implemented in the mapper.

In practice, there are various technical problems that need to be addressed for learning and testing of protocol implementations to run smoothly. The SUL/T, which is viewed as a black-box, is an independent OS process that needs to be managed. This process might exhibit non-deterministic behavior, or even crash. Both of these behaviors are problematic for many learning algorithms, and thus should be detected and handled appropriately. Additionally, there are often other requirements, such as configuring the SUL/T in a certain way or collecting statistics about learning. Finally, various optimizations should be supported for the learning to be efficient. The following paragraphs describe all these in detail.

Process Management The SUL/T process needs to be managed during the learning or testing procedure. For example, when the learner issues independent queries, it usually expects that the underlying state of the SUL/T is the same. This implies that a fresh SUL/T process needs to be created before each learning query and be destroyed afterwards. Often, starting and stopping these processes depend on how the implementation is set up. For example, sometimes the SUL/T process cannot be launched directly, but only through a launch server with specific commands. PSF supports all these cases.

Handling Non-Determinism Sometimes, the SUL/T might (seem to) exhibit non-deterministic behaviors. Non-determinism is problematic for learning. If the implementation is deterministic, then the inconsistency often originates from the user specified response timeout, namely, how long the learning component needs to wait for an SUL/T’s response. From the operating system’s perspective, the protocol state fuzzing tool and the SUL/T are two independent processes that communicate with each other. However, the communication’s speed can fluctuate due to various reasons such as resource contention, (local) network latency, and system load. Thus, the awaited responses might sometimes be delayed more than the wait period, and make the SUL/T appear to be non-deterministic. PSF needs to detect and handle cases when input queries lead to non-determinism. One possible mitigation is to offer support for repeating such queries multiple times (e.g., five or seven), and then use *majority voting* to return the most probable

response to the learner [9,23]. As another possible mitigation, PSF also supports a *timing* mode that allows measuring the time it usually takes for the system to start, respond or execute specific input queries. This information can then be used to calibrate the values for the various timeouts.

Configuration and Diagnostics In addition to support for logging, PSF provides hooks to configure various parameters of the learner, such as the learning duration, the maximum number of queries to be performed, etc. This requires a tracking mechanism that can enforce those limits and gracefully terminate the procedure. A similar mechanism can be used to gather runtime statistics for analysis. Some useful metrics are the time each of the phases of hypothesis construction and validation took, the number of queries each performed, the number of individual inputs sent, the intermediate counterexamples found, etc.

Optimizations PSF comes with support for optimizations that can speed up learning. The first is a caching mechanism to store the input queries and the corresponding outputs. Assuming a deterministic SUL, the next time the same input query is requested, the cache answers immediately instead of waiting for the SUL. The cache can be used in both hypothesis construction and validation phases, as well as act as a building block for a non-determinism detector. Similarly, if the SUL crashes due to a message in the input query that the SUL does not (properly) handle, a mechanism that detects the crash and immediately returns a custom response can save a lot of learning time. Otherwise, learning has to wait for a timeout for each of the remaining messages in the input query. Finally, in the hypothesis validation phase, an important optimization is to check whether a counterexample is spurious by rerunning it if needed. A spurious counterexample is discarded so that a true counterexample can be found in the same phase, this way reducing the overall learning duration.

3 Design of PROTOCOLSTATE-FUZZER

PSF is a framework that simplifies the implementation of a protocol-specific learning and testing tool. It handles the integration of the learning, mapping, system, and testing components while also dealing with the requirements and challenges described in the previous section. Specifically, PSF provides not only ready-to-use learning and testing components, but also the necessary interfaces to implement the protocol-specific mapping and system components.

Each of PSF’s three modes (learning, testing, and timing) is represented by a different mode component and uses the same abstract input and output messages. For example, testing takes one or more sequences of abstract input messages, and returns the corresponding sequences of abstract output messages or an indication that the SUT crashed. Testing also accepts a (learned) state machine model as a parameter to check whether the output sequences of the model and the SUT differ. Similarly, the timing mode can be used to find the best timeout values, in a given range, that do not cause non-determinism. It can (automatically) search for the best response timeout of the SUL/T, the best startup timeout (wait time for a fresh SUL/T instance to be ready to respond),

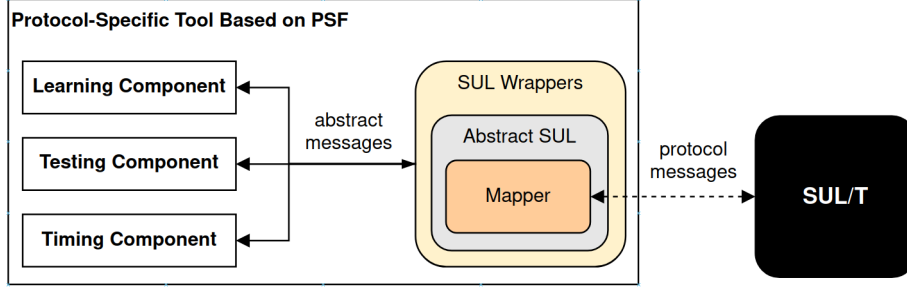


Fig. 1. The architecture of a tool for a specific network protocol based on PSF. The left part of the figure shows components corresponding to the three modes of use that PSF supports: learning, testing, and timing. The middle part shows PSF interfaces and elements that need to be implemented by the tool developer. The Mapper (orange) is protocol-specific, and is wrapped by the SUL Wrappers (yellow) and Abstract SUL (grey) of PSF’s system component. The solid arrows represent same process in-memory communication, while the dashed arrow denotes inter-process network communication.

or the best additional time to wait after input messages are sent. This additional wait time is useful when specific sequences of input message (compared to the other input messages) invoke more work on the SUL/T’s side and thus an extra wait is needed on top of the common response wait time.

The architecture of a protocol-specific tool based on PSF is shown in Fig. 1. The tool brings together the mode components it wants to implement, the mapping component, represented by the *Mapper*, and the system component, represented by the *Abstract SUL* wrapped in the *SUL Wrappers* that provide extra functionality. The mapping component is nested inside the system component, because both rely on the protocol to be learned/tested. PSF provides the necessary interfaces of *Abstract SUL* and *Mapper*, as well as a plethora of ready-to-use *SUL Wrappers* that help with the SUL/T communication.

Figure 2 shows the architecture of PSF’s learning component. The *State Fuzzer* (blue) is responsible for coordinating learning and alternating between its two phases: *hypothesis construction* and *hypothesis validation*. The upper part, which depicts the hypothesis construction phase, contains the *Learner* (green) and its corresponding *Membership SUL Oracle* (yellow-white). The latter consists of the abstract *SUL Oracle* (white), which propagates abstract messages to the SUL through the mapping component (after the right end), and several wrapping *Helper Oracles* (yellow), which provide extra functionality. During hypothesis construction, the *Learner* performs membership queries to its *Membership SUL Oracle*, records its responses, and builds a hypothesis of the state machine model. The lower part depicts the hypothesis validation phase. It consists of a *Validator* (red) and a corresponding *Equivalence SUL Oracle* (yellow-white). During this phase, the *Validator* uses possibly many equivalence algorithms to find a counterexample to the current hypothesis by making equivalence queries to its *Equivalence SUL Oracle*. When a counterexample C is found, the *State*

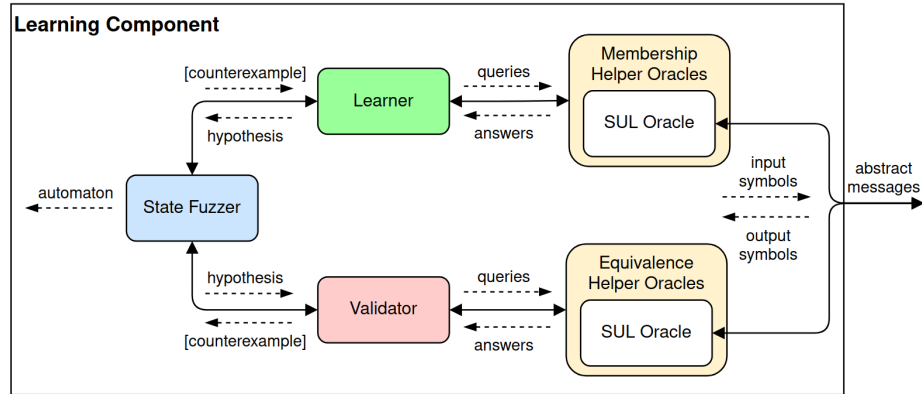


Fig. 2. The architecture of PSF’s learning component with annotated data flow. The upper part represents the hypothesis construction phase, while the lower represents the hypothesis validation phase. The learning component communicates with the SUL via queries, which are sequences of input and output symbols (abstract messages).

Fuzzer restarts the hypothesis construction phase, and provides C to the *Learner* to refine its current hypothesis.

4 Implementation of PROTOCOLSTATE-FUZZER

In this section, we describe the most important aspects of PSF’s implementation.

4.1 Learning Support

PSF is implemented in Java. For state machine learning, it currently hooks with two active automata learning (AAL) libraries. The first, LearnLib [20,15], comes with AAL algorithms that learn Finite State Machines (FSMs) models of the abstract behavior of the SUL, most notably Mealy Machines. The second library, RALib [3,7], provides algorithms for learning Register Automata (i.e. Extended FSMs), which allows learning state machine models with data parameters (e.g., sequence numbers in network packets), as opposed to input and output alphabets with plain symbols. Since PSF’s learning API is already parametric on the AAL algorithm to use, using Java generics, extending the tool to support another learning library accessible from Java should not be that difficult.

All PSF modes run over abstract input and output alphabets. The input alphabet is configurable during runtime. For example, when we want some input symbols to not affect the learning procedure, these can simply be excluded from the input alphabet. In contrast, the output alphabet by default always consists of all the output symbols.

The currently supported membership and equivalence algorithms of each format are shown in Table 1; they are selectable via options. Equivalence algorithms

Table 1. Membership and equivalence algorithms available in PSF. Algorithms for learning Mealy Machines are offered by LearnLib, and for learning Register Automata by RALib. Colored in blue are custom algorithms implemented in PSF.

	Membership Algorithms	Equivalence Algorithms
Mealy Machines (FSMs)	L^* [1]	Random Walk
	TTT [19]	W Method [5]
	Rivest-Schapire [30]	WP Method [16]
	Kearns-Vazirani [21]	Random WP Method Sampled Tests WP Sampled Tests
Register Automata (EFSMs)	SL^* [3]	IO Random Walk
	SL^λ [7]	

shown in blue are custom implementations of PSF. Specifically, the Random WP Method algorithm is a variant of WP Method [16], which uses sampling, rather than enumeration, to create test queries. To accelerate the hypothesis validation phase, this algorithm comes with support for parallelization over a user-specified number of threads. The Sampled Tests algorithm allows the user to provide a list of equivalence queries that are always performed sequentially. Lastly, the WP Sampled Tests algorithm is a hybrid of the other two; it closely follows the Random WP Method, but samples from a specified list of queries a random suffix that constitutes the middle sequence of the new test query.

4.2 System Aspects

PSF tackles the practical challenges detailed in Section 2 using the different helper classes colored in yellow, namely the *SUL Wrappers* in Fig. 1 and the *Membership Helper Oracles* and *Equivalence Helper Oracles* in Fig. 2. All of these classes are integrated in the learning and testing procedure via implementing the necessary LearnLib interfaces. The Helper Oracles process entire sequences of inputs and outputs (sequences of abstract symbols). In contrast, the *SUL Wrappers* process only individual input and output symbols, but also expose `pre` and `post` methods to be used before and after a single query.

Process Management PSF provides ready-to-use *SUL Wrappers* that appropriately handle the SUL/T process. The first is the `SULProcessWrapper`, which is responsible for handling the SUL/T process in one of two distinct points: either create it before and destroy it after each new query, or create it once in the beginning of the current procedure and destroy it in the end. The second wrapper is the `SULAdapterWrapper`, which can be optionally enabled when the SUL/T can only start through a launch server. The job of this wrapper is to send the necessary commands to the server in order for the SUL/T process to start or stop. Since this is protocol-specific, `SULAdapterWrapper` is based on an `SULAdapter` interface, which needs to be implemented by the tool that uses PSF.

Handling Non-Determinism PSF handles the non-deterministic behavior of the SUL/T via two *Membership Helper Oracles*. As mentioned, these have access to the input and output queries. The first is the `MultipleRunsSULOracle`, optionally enabled, that proceeds in two phases. First, it runs every query a specified number of times and checks the outputs. If there are differing answers, this oracle optionally starts the second phase, probabilistic sanitization. This entails running the input query multiple times until an answer appears with a specified acceptable likelihood. If probabilistic sanitization is disabled or fails to find an acceptable answer, the learning stops due to detecting non-determinism, and a comprehensive error report is generated. The only problem with the `MultipleRunsSULOracle` is that it will always run the input query many times, increasing the learning duration. To overcome this, PSF also provides a `NonDeterminismRetryingSULOracle`, an extension of the previous one with an additional phase in the beginning. This phase runs the input query only once, and utilizes a shared input-output query cache to check whether the received answer differs from the cached one (if any). If they differ, it proceeds with the two phases as previously. In most cases, this oracle runs the input query once, checks the cache, and returns the cached answer immediately afterwards.

Configuration and Diagnostics The configuration issues regarding total learning time or total number of learning queries are handled by two *SUL Wrappers* that keep track of the respective quantities. As wrappers, they handle individual input symbols, but can calculate the start and end of a new query via their `pre` and `post` methods. Similarly, diagnostics about the input queries or symbols are obtained via LearnLib’s `ResetCounterSUL` and `SymbolCounterSUL`, which are incorporated as *SUL Wrappers* as well. Moreover, PSF keeps track of various other statistics regarding the learning progress, such as runtime, found counterexamples, sizes of intermediate hypotheses, etc.

Optimizations The caching mechanism mentioned in Section 2 is realized in the `CachingSULOracle` that can be used either by the membership or equivalence oracles, as already mentioned in the paragraph handling non-determinism. On the other hand, PSF also supports tracking of the SUL/T’s liveness status through a shared tracker. This is also used by an *SUL Wrapper* called `SULLivenessWrapper`, which answers immediately with a fixed symbol when it detects that the SUL/T process is not alive anymore. Without this immediate response, after the SUL/T has crashed, every remaining input symbol incurs a cost of at least the wait time of the response, which can be significant if the remaining symbols are many. Finally, to optimize the hypothesis validation phase, PSF provides the `CESanitizingSULOracle`, which is an extension of the `MultipleRunsSULOracle` that tries to confirm a potential counterexample using the shared cache and rerunning it if needed.

5 Current Uses of PROTOCOLSTATE-FUZZER

The initial motivation for PSF was to facilitate the development of [EDHOC-Fuzzer](#), an analysis and testing tool for EDHOC clients and servers. EDHOC

(Ephemeral Diffie-Hellman Over COSE) is recent lightweight authenticated key exchange protocol that provides improved security with less overhead for IoT devices [35]. EDHOC-Fuzzer has been built with PSF as its basis. It has been used to analyze three different EDHOC implementations (one written in C, one in Java, and one in Rust) and has found state machine bugs in all of them [32].

The first version of PSF inherited some code from a prototype tool developed by Fiterau et al. [9] to analyze DTLS (Datagram Transfer Layer Security) servers. That prototype was later extended to also handle DTLS clients, and was released as [DTLS-Fuzzer](#) [11]. DTLS-Fuzzer has been used to analyze various different versions of the major implementations of DTLS 1.2 (GnuTLS, JSSE, MbedTLS, OpenSSL, PionDTLS, Scandium, TinyDTLS, wolfSSL, etc.), and has discovered numerous bugs and security vulnerabilities in their code bases [9,12].

Once the API of PSF became stable, DTLS-Fuzzer was adapted to use PSF as its basis, a refactoring effort that not only reduced the code size of DTLS-Fuzzer by several thousand lines of code, but also significantly eased the effort for its maintenance. Recently, DTLS-Fuzzer was extended with the ability to also analyze client and server implementations of the DTLS 1.3 protocol [4].

At the moment of this writing (January 2026), PSF’s code base comprises 15 771 lines of Java code. We note that this number does not include the code sizes of [LearnLib](#) and [RALib](#) which PSF employs as libraries for active automata learning. In contrast, EDHOC-Fuzzer’s code base comprises 8 596 lines of Java code, and DTLS-Fuzzer’s only 3 214. Although the number of lines of code is only a rough indication of the development effort and complexity of a software tool, it should be clear that PSF allows one to offload to a library and reuse a significant portion of the code that is required to build protocol-specific analysis and testing tools. As further corroborating evidence, an M.Sc. student of our group recently built [BLE-Fuzzer](#), an effective protocol state fuzzer for the BLE (Bluetooth Low Energy) protocol using PSF as a basis, some off-the-shelf Python libraries as a mapper, and only 475 lines of Java code [34]. Efforts to build analysis and testing tools for other network protocols (e.g., for Zigbee) using PSF as a basis are currently underway.

We refer to the publications cited in this section for more information on the quantitative aspects of tools that use PSF as a basis.

6 Related Work

Protocol state fuzzing uses model learning [27,38] to generate state machine models of network protocol implementations. In addition to DTLS and EDHOC, protocol state fuzzing has been applied to various network and IoT protocols. In the last decade, researchers have successfully used this technique to learn state machine models and test implementations of e.g., TLS [31,28,22], TCP [10], SSH [14], OpenVPN [6], MQTT [37], IPSec [17], QUIC [29], and Bluetooth devices [26], among others, and uncover a significant number of errors in them. Although all these tools are effective for their (research) purposes, a downside with them is that they are monolithic: they are protocol-specific tools that cannot

be extended or reused easily for other, even similar, protocols. On the contrary, PSF is generic (i.e. protocol-agnostic) and aims to be an extensible framework that is reusable as a library.

A tool which is similar to PSF in some ways is **PROGNOSIS**, a framework that offers closed-box automated learning and analysis of models of network protocol implementations [8]. PROGNOSIS’s architecture is also modular, though more involved than PSF’s, requiring its users to specify more components. PROGNOSIS also uses LearnLib for its Learner, but can learn Mealy machines using only one membership algorithm (TTT) and one equivalence algorithm (WP Method). In contrast, PSF provides a variety of membership and equivalence algorithms for Mealy Machines (cf. Table 1), including some custom ones which also come with parallelization support. PROGNOSIS can also learn richer models that include registers (to store, e.g., packet numbers), but these richer models are derived from Mealy machines by using a *synthesis* procedure, instead of interfacing with a general library, such as RALib, and be able to use a set of algorithms that learn directly Register Automata in an efficient way. Functionality-wise, PROGNOSIS provides learning and testing (trace running) modes for server implementations only. In contrast, PSF provides learning, testing, and timing modes for both client and server implementations of a protocol.

7 Conclusion and Future Work

This paper presented the most important aspects of the design and implementation of PSF, a generic, modular, and extensible framework that can be used as a basis for state machine learning and testing of network protocol implementations. Our experience of using PSF is that it already provides the features that are needed to overcome the most practical hurdles and build an effective protocol-specific state fuzzer tool on top of it.

Like most software libraries, PSF’s functionality can be enhanced. One direction is to support additional active automata learning algorithms that exist or may become available in LearnLib, RALib, or other libraries; another is to provide support for *differential testing* of state machine models [2,18]. Currently, PSF’s testing component supports (i) testing/fuzzing a SUT with a set of input queries (e.g., to find assertion violations or crash it in other ways) and (ii) testing an implementation in order to find input sequences that result in diverging outputs from a given state machine model of the system. A natural extension would be to also add the ability to take as input two (or more) state machine models of implementations of the same protocol, and produce input sequences that expose differences in their models. We are currently working towards this.

8 Data-Availability Statement

The latest version of the source code and additional information about PSF is at: <https://github.com/protocol-fuzzing/protocol-state-fuzzer>. In addition, the paper’s artifact [33] can be found at: <https://doi.org/10.5281/zenodo.19831626>.

Acknowledgments. We thank Paul Fiterău for comments on a draft of this paper and for his work on DTLS-Fuzzer whose architecture and code base have provided inspiration for the first version of PSF. This research was partially funded by grant 621-2023-05363 from the Swedish Research Council (Vetenskapsrådet).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Angluin, D.: Learning regular sets from queries and counterexamples. *Information and Computation* **75**(2), 87–106 (1987). [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6)
2. Argyros, G., Stais, I., Jana, S., Keromytis, A.D., Kiayias, A.: SFADiff: Automated evasion attacks and fingerprinting using black-box differential automata learning. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. pp. 1690–1701. ACM (Oct 2016). <https://doi.org/10.1145/2976749.2978383>
3. Cassel, S., Howar, F., Jonsson, B., Steffen, B.: Learning extended finite state machines. In: Giannakopoulou, D., Salaün, G. (eds.) *Software Engineering and Formal Methods*. LNCS, vol. 8702, pp. 250–264. Springer International Publishing, Cham (Sep 2014). https://doi.org/10.1007/978-3-319-10431-7_18
4. Chen, N., Fiterău-Broștean, P., Sagonas, K.: State fuzzing DTLS 1.3 implementations. In: *2025 IEEE Conference on Standards for Communications and Networking*. pp. 426–432. CSCN 2025, IEEE (Sep 2025). <https://doi.org/10.1109/CSCN67557.2025.11230763>
5. Chow, T.S.: Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering* **SE-4**(3), 178–187 (1978). <https://doi.org/10.1109/TSE.1978.231496>
6. Daniel, L.A., Poll, E., de Ruiter, J.: Inferring OpenVPN state machines using protocol state fuzzing. In: *IEEE European Symposium on Security and Privacy (EuroS&P) Workshops*. pp. 11–19. IEEE (Apr 2018). <https://doi.org/10.1109/EuroSPW.2018.00009>
7. Dierl, S., Fiterău-Broștean, P., Howar, F., Jonsson, B., Sagonas, K., Tåquist, F.: Scalable tree-based register automata learning. In: Finkbeiner, B., Kovács, L. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*. LNCS, vol. 14571, pp. 87–108. Springer Nature Switzerland, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57249-4_5
8. Ferreira, T., Brewton, H., D’Antoni, L., Silva, A.: Prognosis: Closed-box analysis of network protocol implementations. In: *Proceedings of the ACM SIGCOMM 2021 Conference*. pp. 762–774. SIGCOMM ’21, ACM, New York, NY, USA (Aug 2021). <https://doi.org/10.1145/3452296.3472938>
9. Fiterău-Broștean, P., Jonsson, B., Merget, R., de Ruiter, J., Sagonas, K., Somorovsky, J.: Analysis of DTLS implementations using protocol state fuzzing. In: *29th USENIX Security Symposium*. pp. 2523–2540. USENIX Security 20, USENIX Association (Aug 2020), <https://www.usenix.org/conference/usenixsecurity20/presentation/fiterau-brosteau>

10. Fiterău-Broștean, P., Janssen, R., Vaandrager, F.W.: Combining model learning and model checking to analyze TCP implementations. In: Computer Aided Verification - 28th International Conference, CAV 2016, Proceedings. LNCS, vol. 9780, pp. 454–471. Springer (Jul 2016). https://doi.org/10.1007/978-3-319-41540-6_25
11. Fiterău-Broștean, P., Jonsson, B., Sagonas, K., Tăquist, F.: DTLs-Fuzzer: A DTLs protocol state fuzzer. In: 15th IEEE Conference on Software Testing, Verification and Validation. pp. 456–458. ICST 2022, IEEE (Apr 2022). <https://doi.org/10.1109/ICST53961.2022.00051>
12. Fiterău-Broștean, P., Jonsson, B., Sagonas, K., Tăquist, F.: Automata-based automated detection of state machine bugs in protocol implementations. In: 30th Annual Network and Distributed System Security Symposium. NDSS 2023, The Internet Society (Feb-Mar 2023). <https://doi.org/10.14722/ndss.2023.23068>
13. Fiterău-Broștean, P., Jonsson, B., Sagonas, K., Tăquist, F.: SMBugFinder: An automated framework for testing protocol implementations for state machine bugs. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1866–1870. ISSTA 2024, ACM, New York, NY, USA (Sep 2024). <https://doi.org/10.1145/3650212.3685310>
14. Fiterău-Broștean, P., Lenaerts, T., de Ruiter, J., Poll, E., Vaandrager, F.W., Verleg, P.: Model learning and model checking of SSH implementations. In: Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software. pp. 142–151. SPIN 2017, ACM, New York, NY, USA (Jul 2017). <https://doi.org/10.1145/3092282.3092289>
15. Frohme, M., Howar, F., Steffen, B.: LearnLib: 10 years later. In: Piskac, R., Rakić, Z. (eds.) Computer Aided Verification - 37th International Conference, CAV 2025. LNCS, vol. 15934, pp. 141–160. Springer Nature Switzerland, Cham (Jul 2025). https://doi.org/10.1007/978-3-031-98685-7_7
16. Fujiwara, S., von Bochmann, G., Khendek, F., Amalou, M., Ghedamsi, A.: Test selection based on finite state models. IEEE Transactions on Software Engineering **17**(6), 591–603 (1991). <https://doi.org/10.1109/32.87284>
17. Guo, J., Gu, C., Chen, X., Wei, F.: Model learning and model checking of IPsec implementations for Internet of Things. IEEE Access **7**, 171322–171332 (Nov 2019). <https://doi.org/10.1109/ACCESS.2019.2956062>
18. Hussain, S.R., Karim, I., Ishtiaq, A.A., Chowdhury, O., Bertino, E.: Noncompliance as deviant behavior: An automated black-box noncompliance checker for 4G LTE cellular devices. In: Kim, Y., Kim, J., Vigna, G., Shi, E. (eds.) Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 1082–1099. CCS '21, ACM (Nov 2021). <https://doi.org/10.1145/3460120.3485388>
19. Isberner, M., Howar, F., Steffen, B.: The TTT algorithm: A redundancy-free approach to active automata learning. In: Bonakdarpour, B., Smolka, S.A. (eds.) Runtime Verification: 5th International Conference, RV 2014, Proceedings. LNCS, vol. 8734, pp. 307–322. Springer, Cham (Sep 2014). https://doi.org/10.1007/978-3-319-11164-3_26
20. Isberner, M., Howar, F., Steffen, B.: The open-source LearnLib - A framework for active automata learning. In: Kroening, D., Pasareanu, C.S. (eds.) Computer Aided Verification - 27th International Conference, CAV 2015. LNCS, vol. 9206, pp. 487–495. Springer (Jul 2015). https://doi.org/10.1007/978-3-319-21690-4_32
21. Kearns, M.J., Vazirani, U.: An Introduction to Computational Learning Theory. The MIT Press (08 1994). <https://doi.org/10.7551/mitpress/3897.001.0001>
22. Maehren, M., Erinola, N., Merget, R., Schwenk, J., Somorovsky, J.: Towards internet-based state learning of TLS state machines. In: 34th USENIX Security

- ity Symposium. pp. 7097–7116. USENIX Security 25, Seattle, WA (Aug 2025), <https://www.usenix.org/conference/usenixsecurity25/presentation/maehren>
23. Moon, S.J., Helt, J., Yuan, Y., Bieri, Y., Banerjee, S., Sekar, V., Wu, W., Yannakakis, M., Zhang, Y.: Alembic: Automated model inference for stateful network functions. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19). pp. 699–718. USENIX Association (Feb 2019), <https://www.usenix.org/conference/nsdi19/presentation/moon>
 24. Muškardin, E., Aichernig, B.K., Pill, I., Pferscher, A., Tappler, M.: AALpy: An active automata learning library. In: Hou, Z., Ganesh, V. (eds.) Automated Technology for Verification and Analysis – 19th International Symposium, ATVA 2021. LNCS, vol. 12971, pp. 67–73. Springer International Publishing, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-88885-5_5
 25. Peled, D.A., Vardi, M.Y., Yannakakis, M.: Black box checking. *J. Autom. Lang. Comb.* **7**(2), 225–246 (2002). <https://doi.org/10.25596/jalc-2002-225>
 26. Pferscher, A., Aichernig, B.K.: Fingerprinting and analysis of Bluetooth devices with automata learning. *Formal Methods Syst. Des.* **61**(1), 35–62 (2022). <https://doi.org/10.1007/S10703-023-00425-Y>
 27. Raffelt, H., Merten, M., Steffen, B., Margaria, T.: Dynamic testing via automata learning. *Int. J. Softw. Tools Technol. Transf.* **11**(4), 307–324 (Oct 2009). <https://doi.org/10.1007/s10009-009-0120-7>
 28. Rasoamanana, A.T., Levillain, O., Debar, H.: Towards a systematic and automatic use of state machine inference to uncover security flaws and fingerprint TLS stacks. In: 27th European Symposium on Research in Computer Security (ESORICS). LNCS, vol. 13556, pp. 637–657. Springer, Cham (Sep 2022). https://doi.org/10.1007/978-3-031-17143-7_31
 29. Rasool, A., Alpár, G., de Ruiter, J.: State machine inference of QUIC (2019). <https://doi.org/10.48550/arXiv.1903.04384>
 30. Rivest, R.L., Schapire, R.E.: Inference of finite automata using homing sequences. *Information and Computation* **103**(2), 299–347 (1993). <https://doi.org/10.1006/inco.1993.1021>
 31. de Ruiter, J., Poll, E.: Protocol state fuzzing of TLS implementations. In: 24th USENIX Security Symposium. pp. 193–206. USENIX Security 15, USENIX Association, Washington, D.C. (Aug 2015), <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter>
 32. Sagonas, K., Typaldos, T.: EDHOC-Fuzzer: An EDHOC protocol state fuzzer. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1495–1498. ISSTA 2023, ACM, New York, NY, USA (Jul 2023). <https://doi.org/10.1145/3597926.3604922>
 33. Sagonas, K., Typaldos, T.: Artifact for CAV’2026 tool paper “PSF: A generic and extensible framework for protocol state fuzzing” (Apr 2026), <https://doi.org/10.5281/zenodo.19831626>
 34. Schnor, H.: Using Protocol State Fuzzing to Test Bluetooth Low Energy Implementations. Master’s thesis, Department of Information Technology, Uppsala University, Sweden (Aug 2025), <https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-565565>
 35. Selander, G., Mattsson, J.P., Palombini, F.: Ephemeral Diffie-Hellman Over COSE (EDHOC). RFC 9528 (Mar 2024). <https://doi.org/10.17487/RFC9528>, <https://www.rfc-editor.org/info/rfc9528>
 36. Steffen, B., Howar, F., Merten, M.: Introduction to active automata learning from a practical perspective. In: Bernardo, M., Issarny, V. (eds.) Formal Methods for Eternal Networked Software Systems: 11th International School on Formal Methods for

- the Design of Computer, Communication and Software Systems, SFM 2011, Advanced Lectures, LNCS, vol. 6659, pp. 256–296. Springer, Berlin, Heidelberg (Jun 2011). https://doi.org/10.1007/978-3-642-21455-4_8
37. Tappler, M., Aichernig, B.K., Bloem, R.: Model-based testing IoT communication via active automata learning. In: IEEE International conference on software testing, verification and validation. pp. 276–287. ICST 2017, IEEE (Mar 2017). <https://doi.org/10.1109/ICST.2017.32>
38. Vaandrager, F.: Model learning. Commun. ACM **60**(2), 86–95 (Jan 2017). <https://doi.org/10.1145/2967606>